

Importaciones y exportaciones de datos XML



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: enero de 2003.

Fecha de la última publicación: diciembre de 2013.

Tabla de contenidos

1 XML y Tecnología Meta4: importaciones y exportaciones de datos

■ Introducción	6
■ XML y tecnología Meta4	7
XML: conceptos generales	7
Document Type Definition (DTD)	8
Componentes de una DTD	10
Elementos	10
Atributos	11
Entidades	12
DTD/XML Schema en relación con Meta4Objects	13
Correspondencia de elementos DTD/XML Schema con Meta4Objects	13
Elementos simples y compuestos	15
Atributos	18
Elementos de la DTD/XML Schema sin correspondencia en un Meta4Object	19
Metodología de diseño de Meta4Objects sobre DTD/XML Schema	20
Reglas de correspondencia entre un Meta4Object y una DTD/XML Schema	21
Correspondencia a nivel de Meta4Object	21
Herencia	22
Correspondencia a nivel de nodo	23
Correspondencia de elementos	23
Mapeos por defecto sin persistencia	24
■ XML y datos	29
■ Meta4Objects en procesos de importación	32
Requerimientos previos a la fase de importación	32
Descripción del proceso de importación	32
Procedimiento para ignorar elementos XML sin correspondencia	36
■ Meta4Objects en procesos de exportación	38
Requerimientos previos a la fase de exportación	38
Descripción del proceso de exportación	39

XML y Tecnología Meta4: importaciones y exportaciones de datos

Acerca de este documento

El objetivo de este documento es presentar las posibilidades que implanta la aplicación para integrarse con aplicaciones externas a través de archivos en formato XML. A su vez, ofrece los mecanismos y requerimientos necesarios para diseñar procesos de importación y exportación basados en el intercambio de datos e información estructurada. Desde esta perspectiva, se ofrecerán algunas recomendaciones de diseño que permitan procesar, intercambiar y reutilizar datos desde entornos informáticos heterogéneos.

Se recomienda un conocimiento previo del modelo jerárquico de diseño de Meta4Objects implantado en la aplicación, así como unas nociones básicas sobre los elementos integrantes y el lenguaje de marcas, que admiten los archivos sujetos a la especificación XML.

Introducción

Meta4 PeopleNet implanta su funcionalidad mediante el concepto de Meta4Objects; una representación estructurada de datos que hace un uso extensible de toda la tecnología orientada a objetos. Los desarrollos basados en el estándar XML pretenden resolver ciertos problemas que surgen en ocasiones con la integración de datos y la portabilidad de éstos sobre otros entornos distintos a los que gestiona la tecnología Meta4.

En este sentido, estas aplicaciones Meta4 tratan de integrar y explotar las posibilidades que ofrece el lenguaje XML para el procesamiento y el intercambio de datos desde una doble perspectiva. Por un lado, pretende ejecutar procesos de importación de datos. Es decir, permitir que la aplicación sea capaz de interpretar y procesar archivos XML externos; y por otro lado permitir ejecutar procesos de exportación de datos. Es decir, permitir que se ejecute un proceso que extraiga datos desde objetos de Meta4 PeopleNet y los transforme en archivos estructurados con extensión .xml.

Desde esta perspectiva, parece evidente que el principal requerimiento tecnológico es adecuar el modelo jerárquico de los objetos de Meta4PeopleNet (Meta4Objects) con el lenguaje etiquetado de marcas de los archivos XML, o más concretamente de la DTD (*Document Type Definition*) o XML Schema, que define la estructura del archivo XML que almacena los datos. Esta correspondencia de elementos permitirá que un proceso de validación garantice el procesamiento correcto de los elementos, es decir, qué elementos de un Meta4Object tienen correspondencia con qué elementos o etiquetas de un archivo XML. Se han establecido unas reglas básicas de correlación de elementos, pero no existe un mecanismo automático de generación de Meta4Objects a partir de la DTD/XML Schema. La correspondencia de elementos se explica en el apartado *XML y Tecnología Meta4*.

La correspondencia de elementos de un Meta4Object y una DTD/XML Schema constituyen la base estructural de los procesos de importación y exportación. Desde el punto de vista de diseño, primero será necesario construir una DTD/XML Schema y, a continuación, inferir a partir de ésta la estructura de un Meta4Object. La importación y exportación de datos se parametriza desde los propios Meta4Objects referenciando unos componentes específicamente desarrollados para estos procesos. Éstos se explican en los apartados *Meta4Objects en procesos de importación* y *Meta4Objects en procesos de exportación*, respectivamente.

Finalmente, es necesario considerar que la aplicación modulariza el diseño de las entidades de negocio en múltiples objetos que almacenan separadamente distintos tipos de dato. El modelo funcional de transferencia de datos obliga por tanto a diseñar innumerables DTD/XML Schemas para poder desarrollar procesos de importación y exportación distintos.

XML y tecnología Meta4

El nivel de aplicación de Meta4 PeopleNet se basa en el concepto de Meta4Objects, característico de la tecnología Meta4. Un *Meta4Object* es una representación jerárquica de datos, donde no sólo se encapsulan datos y procesos, sino información y los mecanismos necesarios para procesarla. Un documento XML está formado por una serie de elementos que identifican los contenidos informativos que contiene y hacen explícita su estructura. La estructura de un documento se define en un documento especial llamado DTD (*Document Type Definition*) o XML Schema.

Según estas definiciones, tanto el modelo de objetos que identifican las entidades de negocio que gestiona la aplicación (Meta4Objects), como las DTD/XML Schemas, basan su definición en una representación de la estructura. Por otro lado, un documento o archivo XML basa su definición en una representación del contenido siempre sujeta a las restricciones implícitas de la DTD/XML Schema.

En resumen, la estructura de un Meta4Object y una DTD/XML Schema son similares desde el punto de vista de las restricciones que aplican en la manipulación de los datos. Sobre la estructura de un Meta4Object y de una DTD/XML Schema, se encapsulará una capa de datos que procesará el contenido informativo de ambas.

XML: conceptos generales

DTD: una DTD (definición del tipo de documento) es una colección de declaraciones XML que definen la estructura reglamentaria, los elementos y los atributos que están disponibles para su uso en documentos o en objetos que cumplan con las restricciones implícitas de la propia DTD.

XML Schema: el XML Schema es un lenguaje de esquema utilizado para describir la estructura y las restricciones de los contenidos de los documentos XML de una forma muy precisa, más allá de las normas sintácticas impuestas por el propio lenguaje XML. Se consigue así una percepción del tipo de documento con un nivel alto de abstracción.

Atributo: es un parámetro de un elemento declarado en la DTD. El tipo de un atributo y su rango de valores, incluyendo la posibilidad de un valor por defecto, se definen en la DTD.

Documento: es una cadena de datos que, tras ser combinada con cualquier otra cadena a la que se reference, queda estructurada de manera que porte información en elementos organizados tal y como se especifica en la correspondiente DTD. Se utiliza indistintamente este término o el de archivo XML.

Elemento: es una unidad estructural de un documento que ha sido declarada en la DTD. El modelo de contenidos del elemento está definido en la DTD, y la explicación adicional puede especificarse en la descripción comentada del elemento.

Entidades: una entidad permite crear documentos modulares. Un documento es modular cuando su contenido se reparte en dos o más archivos, aunque se visualizan de forma compacta. Las entidades se utilizan con cuatro propósitos principales:

- Inserción de caracteres especiales en los documentos
- Utilización de cadenas de texto alternativas en lugar de términos o frases complejas de escribir que se desconocen en el momento de redactar el documento
- Inserción de componentes multimedia, imágenes o cualquier documento que no sean datos XML en un documento XML
- Modularización de documentos

Hojas de estilo: una hoja de estilo permite transformar y formatear archivos XML. Normalmente, los documentos XML están vinculados a hojas de estilo que contienen la información relativa a cómo se deben formatear los documentos. El término *formato* se refiere a las características con las cuales se presenta un documento en pantalla o impresora: tipo de letra, interlineado, márgenes, etcétera.

En el diseño de hojas de estilo para documentos en XML existen dos alternativas: utilizar *hojas de estilo CSS (Cascading Style Sheets)*, heredadas del lenguaje HTML para el cual se diseñaron, o utilizar *hojas de estilo XSL (eXtensible Stylesheets Language)*, específicamente diseñadas para documentos XML.

En el diseño de los procesos de importación y exportación de datos en Meta4 PeopleNet no se requiere formatear los datos de ningún modo especial, por lo que a lo largo del documento se obvia el tratamiento de hojas de estilo.

Document Type Definition (DTD)

El conjunto de elementos que se pueden incluir en un documento de un tipo determinado, el orden en el que deben aparecer y cómo deben anidarse se explican en la llamada *definición de tipo de documento*, o *DTD*. Contiene además las reglas sintácticas que deben cumplir los documentos de un tipo determinado.

En una DTD se indica:

- Qué elementos pueden ser utilizados en un tipo de documento específico
- Cuáles son obligatorios y cuáles no

- Cuáles son repetibles y cuáles no
- En qué orden deben aparecer
- Cómo deben anidarse los elementos que conforman un documento

La sintaxis de la DTD, además de cumplir todas las especificaciones desarrolladas por el W3C, debe permitir identificar la correspondencia estructural de sus atributos con los elementos de un Meta4Object.

La DTD puede residir en un archivo externo, quizá compartido por varios documentos, o estar contenida en el propio documento o archivo XML como parte de su declaración.

EJEMPLO DE DECLARACIÓN DE UNA DTD EN EL PROPIO ARCHIVO XML:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE STD_PERSON SYSTEM "c:\xmlmind\STD_PERSON.dtd">
<STD_PERSON>
  <REC_PERSON>
    <STD_ID_PERSON>PERSON01</STD_ID_PERSON>
    <STD_N_FIRST_NAME>Patricia</STD_N_FIRST_NAME>
    <STD_N_FAMILY_NAME1>Fernandez</STD_N_FAMILY_NAME1>
    <STD_N_USUAL_NAME>Patricia</STD_N_USUAL_NAME>
    <STD_ID_GENDER>M</STD_ID_GENDER>
    <STD_SALUTATION>Mrs</STD_SALUTATION>
    <STD_DT_BIRTH>1975-10-01</STD_DT_BIRTH>
    <STD_ID_COUNTRY>Espana</STD_ID_COUNTRY>
    <STD_ID_GEO_DIV>Salamanca</STD_ID_GEO_DIV>
    <STD_ID_SUB_GEO_DIV>Ciudad Rodrigo</STD_ID_SUB_GEO_DIV>
    <STD_COMMENT/>
    <SCO_HOME_PAGE>http://patricia.meta4.es</SCO_HOME_PAGE>
    <STD_ADDRESS>
      <STD_ID_PERSON>PERSON01</STD_ID_PERSON>
      <STD_OR_ADDRESS>Roma 3, Esc2 1A</STD_OR_ADDRESS>
      <STD_ADDRESS_LINE_1>Roma 3</STD_ADDRESS_LINE_1>
      <STD_ADDRESS_LINE_2>Escalera 2</STD_ADDRESS_LINE_2>
      <STD_ADDRESS_LINE_3>Piso 1</STD_ADDRESS_LINE_3>
      <STD_ADDRESS_LINE_4>Puerta A</STD_ADDRESS_LINE_4>
      <STD_ID_COUNTRY>Espana</STD_ID_COUNTRY>
      <STD_ID_GEO_DIV>Salamanca</STD_ID_GEO_DIV>
      <STD_ID_SUB_GEO_DIV>Ciudad Rodrigo</STD_ID_SUB_GEO_DIV>
      <STD_ID_GEO_PLACE>12</STD_ID_GEO_PLACE>
      <STD_ZIP_CODE>47678</STD_ZIP_CODE>
```

```
</STD_ADDRESS>
```

EJEMPLO DE DECLARACIÓN DE LA SINTAXIS DE UNA DTD EXTERNA:

```
<!ELEMENT STD_PERSON (REC_PERSON*)>
<!ELEMENT REC_PERSON (STD_ID_PERSON, STD_N_FIRST_NAME,
STD_N_FAMILY_NAME1, STD_N_USUAL_NAME?, STD_ID_GENDER?,
STD_SALUTATION?, STD_DT_BIRTH?, STD_ID_COUNTRY?, STD_ID_GEO_DIV?,
STD_ID_SUB_GEO_DIV?, STD_COMMENT?, SCO_PHOTO?, SCO_HOME_PAGE?,
STD_ADDRESS*, STD_EMAIL*)>
<!ELEMENT STD_ID_PERSON (#PCDATA)>
<!ELEMENT STD_N_FIRST_NAME (#PCDATA)>
<!ELEMENT STD_N_FAMILY_NAME1 (#PCDATA)>
<!ELEMENT STD_N_USUAL_NAME (#PCDATA)>
<!ELEMENT STD_ID_GENDER (#PCDATA)>
<!ELEMENT STD_ID_SALUTATION (#PCDATA)>
<!ELEMENT STD_DT_BIRTH (#PCDATA)>
<!ELEMENT STD_ID_COUNTRY (#PCDATA)>
<!ELEMENT STD_ID_GEO_DIV (#PCDATA)>
<!ELEMENT STD_ID_SUB_GEO_DIV (#PCDATA)>
<!ELEMENT STD_COMMENT (#PCDATA)>
<!ELEMENT SCO_PHOTO (#PCDATA)>
<!ELEMENT SCO_HOME_PAGE (#PCDATA)>
```

Componentes de una DTD

Elementos

Los elementos, que constituyen la base de las marcas XML, se corresponden con los elementos estructurales de un documento y definen su estructura lógica. Las declaraciones de tipo de elemento se encierran entre `<!ELEMENT` y `>`, seguidas por el identificador genérico del elemento que se declara, y, a continuación, la declaración del contenido que aparecerá siempre entre paréntesis.

Un elemento puede tener contenido de tipo **datos** o **#PCDATA**, **elementos** (anida otros elementos), **mixto** (datos y elementos), **vacío** (el elemento no contiene ni datos ni elementos) y **any** (indica que no hay restricciones para su contenido).

Si en la declaración del contenido existen otros elementos, se puede indicar si estos elementos son obligatorios o no, y el orden en el que deben aparecer.

Estas restricciones se indican escribiendo un carácter especial tras el nombre del elemento utilizado.

Tabla 1. Restricciones aplicables en la declaración de contenido

Signo	Descripción
?	Opcional no repetible. El elemento puede aparecer 0 ó 1 vez.
*	Opcional y repetible. El elemento puede aparecer 0 o más veces.
+	Obligatorio y repetible. El elemento debe aparecer 1 o más veces.
	No existe signo. Obligatorio no repetible. El elemento debe aparecer 1 vez.
,	La coma denota secuencia.
	Indica opción.

Ejemplo:

```
<!ELEMENT autor (nombre, apellido+)>
<!ELEMENT catalogo (producto+)>
<!ELEMENT producto (especificaciones+, opciones?, precio+, notas?)>
<!ELEMENT especificaciones (#PCDATA, step)*>
<!ELEMENT autor (nombre, apellido, posición?, dirección)>
<!ELEMENT contenido ( p | nota )+>
<!ELEMENT front (titulo, res?, claves*, autor+, resumen?)>
```

Atributos

Los atributos permiten agregar información adicional a los elementos para matizar su significado. La principal diferencia entre elementos y atributos es que éstos no pueden contener subatributos y, además, cada uno de los atributos sólo se puede especificar una vez y en cualquier orden.

Las declaraciones de los atributos empiezan con `<! ATTLIST`; a continuación, aparece un conjunto de propiedades que identifican el nombre, el tipo de dato que tiene, un valor por defecto (si existe), y un indicador que señala si son obligatorios o no.

Un atributo puede incluir la siguiente información:

- CDATA: datos de tipo carácter.
- ID: reservado para elementos destino de enlaces.

- IDREF: reservado para elementos origen de enlaces.
- ENTITY: referencias a entidades externas.

Para indicar el carácter obligatorio u opcional de un atributo, se utilizan las palabras reservadas:

- #IMPLIED: el atributo será opcional.
- #REQUIRED: el atributo será obligatorio.
- #FIXED: el atributo es asignado por defecto por el sistema y no es necesario que el autor lo indique.

Ejemplo de declaración de atributos:

```
<!ATTLIST emp fecnac CDATA #IMPLIED>
<!ATTLIST seccion enlace ID #REQUIRED
                        lastupdate CDATA #IMPLIED>
<!ATTLIST empleado fecnacimieto CDATA #REQUIRED
                        personId CDATA #REQUIRED
                        compañía CDATA #FIXED(IBM|Lotus) "IBM">
```

Entidades

Las entidades en las declaraciones XML se utilizan con cuatro propósitos fundamentales:

- Inserción de caracteres especiales en los documentos
- Utilización de cadenas de texto alternativas en lugar de términos o frases complejos de escribir o que se desconocen en el momento de crear el documento
- Inserción de componentes multimedia, imágenes o cualquier documento que no sean datos XML, en un documento XML
- Modularización de los documentos

La declaración de las entidades se realiza mediante la etiqueta <!ENTITY.

Las entidades pueden ser:

- Internas o externas
- Analizadas o no analizadas
- Generales o parámetro

Entidades internas: se utilizan básicamente en la definición de abreviaturas. Son siempre entidades analizadas; es decir, una vez reemplazada la referencia a la entidad por su contenido, pasa a ser parte del documento XML y como tal es analizada por el procesador de XML.

```
<!ENTITY alf "Alien Life Form">
```

```
<texto><titulo>Un día en la vida de un &alf;</titulo></texto>
```

Entidades externas analizadas: el contenido al que hacen referencia se encuentra en cualquier sitio del sistema: otro archivo de la unidad local, una página Web o un objeto de la base de datos. Se declara el contenido mediante los atributos SYSTEM y una URL.

```
<!ENTITY intro SYSTEM "http://....intro.xml">
```

Entidades generales externas no analizadas: *el contenido al que hacen referencia se encuentra en un archivo externo (.gif, .exe) no procesable o interpretable por un parser XML.*

Entidades parámetro internas y externas: sólo pueden usarse en la DTD y no en el documento XML. Se pueden utilizar para agrupar ciertos elementos de la DTD que se repiten con frecuencia. Se referencian con % y ;

```
<!ENTITY % elemento-alf "<!ELEMENT alf (#PCDATA)">
```

DTD/XML Schema en relación con Meta4Objects

En el diseño que se desarrolla en la tecnología Meta4 para procesos de importación y exportación de datos, es un requisito imprescindible parametrizar y construir Meta4Objects que cumplan siempre las restricciones de la DTD/XML Schema. Este tipo de restricciones obligan a establecer una relación entre los atributos de la DTD/XML Schema y los elementos estructurales (nodos, elementos, etcétera) del Meta4Object. Durante la ejecución de los procesos de importación y exportación, se valida la correspondencia de elementos, lo que garantiza el correcto procesamiento de los datos.

La propia sintaxis de la DTD/XML Schema, según lo explicado en los apartados anteriores, determinará el *modus operandi* de construcción de un Meta4Object. Sin embargo, dentro de una DTD es obligatorio restringir el uso a elementos compuestos del tipo #PCDATA and any of... , y a la declaración de entidades (<!ENTITY..>). En ambos casos, son elementos que no tienen posibilidad de correspondencia con ningún objeto dentro del Meta4Object.

En la actualidad, no existe ningún mecanismo para construir un Meta4Object automáticamente infiriendo la estructura de una DTD/XML Schema.

Correspondencia de elementos DTD/XML Schema con Meta4Objects

A continuación se presenta una definición de los distintos elementos de la DTD/XML Schema que se van a considerar, y la correspondiente correlación con los elementos de un Meta4Object. Existen elementos dentro de la DTD/XML Schema que no tienen correspondencia a nivel de Meta4Object.

En el siguiente ejemplo se utilizan las referencias que aparecen en la DTD:

EJEMPLO

```
<!ELEMENT STD_PERSON (REC_PERSON*)>
<!ELEMENT REC_PERSON (STD_ID_PERSON, STD_N_FIRST_NAME,
STD_N_FAMILY_NAME1, STD_N_USUAL_NAME?, STD_ID_GENDER?,
STD_SALUTATION?, STD_DT_BIRTH?, STD_ID_COUNTRY?, STD_ID_GEO_DIV?,
STD_ID_SUB_GEO_DIV?, STD_COMMENT?, SCO_PHOTO?, SCO_HOME_PAGE?,
STD_ADDRESS*, STD_EMAIL*)>
<!ELEMENT STD_ID_PERSON (#PCDATA)>
<!ELEMENT STD_N_FIRST_NAME (#PCDATA)>
<!ELEMENT STD_N_FAMILY_NAME1 (#PCDATA)>
<!ELEMENT STD_N_USUAL_NAME (#PCDATA)>
<!ELEMENT STD_ID_GENDER (#PCDATA)>
<!ELEMENT STD_ID_SALUTATION (#PCDATA)>
<!ELEMENT STD_DT_BIRTH (#PCDATA)>
<!ELEMENT STD_ID_COUNTRY (#PCDATA)>
<!ELEMENT STD_ID_GEO_DIV (#PCDATA)>
<!ELEMENT STD_ID_SUB_GEO_DIV (#PCDATA)>
<!ELEMENT STD_COMMENT (#PCDATA)>
<!ELEMENT SCO_PHOTO (#PCDATA)>
<!ELEMENT SCO_HOME_PAGE (#PCDATA)>
<!ELEMENT STD_ADDRESS (STD_ID_PERSON, STD_OR_ADDRESS, STD_DT_START?,
STD_DT_END?, STD_ID_LOCATION_TYPE?, STD_ADDRESS_LINE_1,
STD_ADDRESS_LINE_2?, STD_ADDRESS_LINE_3?, STD_ADDRESS_LINE_4?,
STD_ID_COUNTRY, STD_ID_GEO_DIV, STD_ID_SUB_GEO_DIV, STD_ID_GEO_PLACE,
STD_ZIP_CODE?)>
<!ELEMENT STD_OR_ADDRESS (#PCDATA)>
<!ELEMENT STD_DT_START (#PCDATA)>
<!ELEMENT STD_DT_END (#PCDATA)>
<!ELEMENT STD_ID_LOCATION_TYPE (#PCDATA)>
<!ELEMENT STD_ADDRESS_LINE_1 (#PCDATA)>
<!ELEMENT STD_ADDRESS_LINE_2 (#PCDATA)>
<!ELEMENT STD_ADDRESS_LINE_3 (#PCDATA)>
<!ELEMENT STD_ADDRESS_LINE_4 (#PCDATA)>
<!ELEMENT STD_ID_GEO_PLACE (#PCDATA)>
<!ELEMENT STD_ZIP_CODE (#PCDATA)>
<!ELEMENT STD_EMAIL (STD_ID_PERSON, STD_OR_MAIL, STD_DT_START?,
STD_DT_END?, STD_ID_MAIL, STD_ID_LOCATION_TYPE?)>
<!ELEMENT STD_OR_MAIL (#PCDATA)>
<!ELEMENT STD_ID_MAIL (#PCDATA)>
<!ELEMENT STD_SALUTATION (#PCDATA)>
```

Elementos simples y compuestos

Básicamente, en una DTD los elementos se agrupan en dos clases: compuestos y simples. Ambos tipos de elemento pueden tener atributos.

Los *elementos compuestos* son los que están formados por otros elementos. Puede ser a través de una estructura de secuencia o de opción. Existe una excepción en la declaración de los elementos compuestos que son los definidos como #PCDATA and any of...; por sus características específicas no se ha establecido ninguna correspondencia con ningún elemento dentro del Meta4Object.

Los *elementos simples* son los que no contienen otros elementos, bien porque establecen su valor (#PCDATA) o porque establecen su valor a vacío (EMPTY).

¿Cómo diseñar e identificar en una DTD elementos simples y elementos compuestos?

Desde el punto de vista sintáctico, los *elementos compuestos* tienen la siguiente forma de declaración:

```
<!ELEMENT Nombre del elemento (declaración del contenido)
```

EJEMPLO

```
<!ELEMENT STD_PERSON (REC_PERSON*)>
<!ELEMENT REC_PERSON (STD_ID_PERSON, STD_N_FIRST_NAME,
STD_N_FAMILY_NAME1, STD_N_USUAL_NAME?, STD_ID_GENDER?,
STD_SALUTATION?, STD_DT_BIRTH?, STD_ID_COUNTRY?, STD_ID_GEO_DIV?,
STD_ID_SUB_GEO_DIV?, STD_COMMENT?, SCO_PHOTO?, SCO_HOME_PAGE?,
STD_ADDRESS*, STD_EMAIL*)>
```

Los elementos compuestos dentro de la DTD se corresponden siempre con nodos de un Meta4Object. El primer elemento compuesto que aparece en la declaración de la DTD siempre es el nodo raíz. A continuación, el resto de los elementos compuestos de la DTD designarán nodos hijos respecto al identificado como nodo raíz. Aunque en el diseño de la DTD/XML Schema es necesario identificar el elemento que se procesará como nodo raíz, la implantación posterior en el diseño de un Meta4Object, no obliga a parametrizar éste como nodo raíz.

Para establecer las dependencias estructurales de los nodos, es decir, determinar qué nodo es padre o hijo y con respecto a quién, se puede decir con carácter general que un elemento de la DTD/XML Schema que hace las veces

de nodo en la declaración del contenido será siempre hijo respecto al elemento que lo contiene.

En el caso del ejemplo anterior:

```
<!ELEMENT REC_PERSON (STD_ID_PERSON, STD_N_FIRST_NAME,
STD_N_FAMILY_NAME1, STD_N_USUAL_NAME?, STD_ID_GENDER?,
STD_SALUTATION?, STD_DT_BIRTH?, STD_ID_COUNTRY?, STD_ID_GEO_DIV?,
STD_ID_SUB_GEO_DIV?, STD_COMMENT?, SCO_PHOTO?, SCO_HOME_PAGE?,
STD_ADDRESS*, STD_EMAIL*)>
```

Los elementos STD_ADDRESS* y STD_EMAIL* están definidos como nodos. Un elemento dentro de una declaración de contenido que aparece con el signo * siempre se procesa como nodo de un Meta4Object. En este caso, ambos elementos son nodos, y dependen estructuralmente del elemento que los contiene REC_PERSON. Son, por lo tanto, nodos hijos respecto al nodo REC_PERSON.

Desde el punto de vista sintáctico, los *elementos simples* tienen dos formas de declaración.

Por un lado, pueden constituir la parte de declaración del contenido de los elementos compuestos. De este modo, podemos decir que todo elemento simple forma parte de un elemento compuesto en función de una estructura de secuencia.

```
<!ELEMENT Nombre del elemento (declaración del contenido o elemento simple)
```

EJEMPLO

```
<!ELEMENT STD_PERSON (REC_PERSON*)>
<!ELEMENT REC_PERSON (STD_ID_PERSON, STD_N_FIRST_NAME,
STD_N_FAMILY_NAME1, STD_N_USUAL_NAME?, STD_ID_GENDER?,
STD_SALUTATION?, STD_DT_BIRTH?, STD_ID_COUNTRY?, STD_ID_GEO_DIV?,
STD_ID_SUB_GEO_DIV?, STD_COMMENT?, SCO_PHOTO?, SCO_HOME_PAGE?,
STD_ADDRESS*, STD_EMAIL*)>
```

Los elementos STD_ID_PERSON, STD_N_FIRST_NAME, etcétera, que aparecen dentro de la declaración de contenido del elemento REC_PERSON, son ejemplos de elementos simples.

Por otro lado, los elementos simples se reconocen fácilmente como los que no contienen otros elementos. Éstos se identifican por los atributos #PCDATA y normalmente por el atributo EMPTY. Existen algunos casos en los que un elemento simple de tipo EMPTY pueden contener subelementos.

EJEMPLO

```
<!ELEMENT STD_OR_ADDRESS (#PCDATA) >
<!ELEMENT STD_DT_START (#PCDATA) >
<!ELEMENT STD_DT_END (#PCDATA) >
<!ELEMENT STD_ID_LOCATION_TYPE (#PCDATA) >
<!ELEMENT STD_ADDRESS_LINE_1 (#PCDATA) >
<!ELEMENT STD_ADDRESS_LINE_2 (#PCDATA) >
<!ELEMENT STD_ADDRESS_LINE_3 (#PCDATA) >
```

Los elementos simples, especialmente los que forman parte de la declaración del contenido, tienen una correspondencia con los elementos de un Meta4Object que está en función de la frecuencia.

Gráficamente, la frecuencia en una DTD y la correspondencia con elementos dentro de un Meta4Object se designa del siguiente modo:

Tabla 2. Correspondencia elementos simples/elementos Meta4Objects

Nombre elemento	Tipo de correspondencia	Descripción
Nombre elemento	Elemento	Elemento sin signo
Nombre elemento?	Elemento	Elemento con signo ?
Nombre elemento*	Nodo	Elemento con signo *
Nombre elemento+	Nodo	Elemento con signo +

EJEMPLO

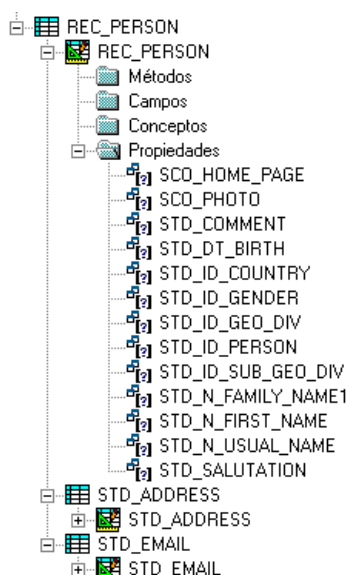
```
<!ELEMENT REC_PERSON (STD_ID_PERSON, STD_N_FIRST_NAME,
STD_N_FAMILY_NAME1, STD_N_USUAL_NAME?, STD_ID_GENDER?,
STD_SALUTATION?, STD_DT_BIRTH?, STD_ID_COUNTRY?, STD_ID_GEO_DIV?,
STD_ID_SUB_GEO_DIV?, STD_COMMENT?, SCO_PHOTO?, SCO_HOME_PAGE?,
STD_ADDRESS*, STD_EMAIL*) >
```

Si la correspondencia de un elemento simple indica que debe procesarse éste como un nodo, dicho nodo aparecerá como hijo respecto al elemento compuesto que lo contiene (que hará las veces de padre). Si la correspondencia de un elemento simple indica que debe procesarse éste como un elemento, dicho elemento aparecerá contenido en el elemento compuesto (que hará las veces de nodo).

Tomando como ejemplo el caso anterior. Existe un elemento compuesto REC_PERSON que se procesará como un nodo del Meta4Object. En la declaración del contenido, aparece un número determinado de elementos simples que se procesarán como elementos del Meta4Object dependiendo estructuralmente del elemento compuesto que los contiene. Hay dos excepciones: los elementos STD_ADDRESS* y STD_EMAIL* se procesarán como nodos del Meta4Object. Este comportamiento se identifica dentro de la DTD/XML Schema por el signo *. Estos nodos se procesarán como nodos hijos respecto al elemento compuesto que los contiene, REC_PERSON, que hará las veces de padre.

Gráficamente, la estructura de componentes del Meta4Object en función de las restricciones implícitas de la DTD/XML Schema sería del siguiente modo:

Figura 1. Meta4Objects en función de la DTD/XML Schema



Atributos

Los elementos simples y los elementos compuestos pueden recibir atributos para matizar su significado.

Desde el punto de vista sintáctico, los atributos tienen la siguiente forma de declaración:

```
<!ATTLIST nombre elemento | nombre atributo | tipo de dato |
obligatorio/opcional>
```

Los elementos que forman parte de la declaración ATTLIST se separan por espacios.

Los tipos de dato que puede recoger un atributo son:

- CDATA: datos de tipo carácter.
- (valor | valor2 | ... | valorn): enumeración.
- ID: reservado para elementos destino de enlaces.
- IDREF: reservado para elementos origen de enlaces.
- ENTITY: referencias a entidades externas.

Para indicar la obligatoriedad o el carácter opcional de un atributo, se utilizan las palabras reservadas #IMPLIED (el atributo será opcional), #REQUIRED (el atributo será obligatorio) o #FIXED (el atributo es asignado por defecto por el sistema y no es necesario que el autor lo indique).

EJEMPLO

```
<!ELEMENT STD_ADDRESS (STD_ID_PERSON, STD_OR_ADDRESS,  
STD_ADDRESS_LINE_1, STD_ADDRESS_LINE_2?, STD_ADDRESS_LINE_3?,  
STD_ADDRESS_LINE_4?, STD_ID_COUNTRY, STD_ID_GEO_DIV,  
STD_ID_SUB_GEO_DIV, STD_ID_GEO_PLACE, STD_ZIP_CODE?) >  
  
<!ATTLIST STD_ADDRESS STD_DT_START CDATA #IMPLIED>  
  
<!ATTLIST STD_ADDRESS STD_DT_END CDATA #IMPLIED>  
  
<!ATTLIST STD_ADDRESS STD_ID_LOCATION_TYPE CDATA #IMPLIED>
```

Los atributos declarados en la DTD/XML Schema se procesan en la aplicación como elementos del Meta4Object. La disposición estructural de estos elementos dependen del modo de declaración dentro de la DTD/XML Schema y del elemento que lo contenga.

Elementos de la DTD/XML Schema sin correspondencia en un Meta4Object

Existen elementos de la DTD/XML Schema procesables con archivos XML estándar que no mantienen una correspondencia con alguno de los elementos del modelo de objetos que gestiona la aplicación. Para garantizar y evitar inconsistencias en el procesamiento de una DTD/XML Schema en procesos de importación y exportación de datos, se trata de limitar la funcionalidad que puede ofrecer aquélla pero sin que afecte a los distintos procesos de ejecución.

En este sentido, se restringe en la DTD el uso de elementos compuestos del tipo #PCDATA and any of... y la declaración de entidades del tipo (internas, externas, parámetro, etcétera). Estrictamente hablando, existe un conjunto de entidades utilizadas para la representación de caracteres especiales

susceptibles de procesarse dentro de la DTD/XML Schema. Son las entidades del tipo *<*, *>*, *&*, *"*, *'*.

Una DTD/XML Schema que admite la transferencia de datos en las aplicaciones Meta4 debe restringir obligatoriamente el uso de elementos del tipo señalado anteriormente.

Metodología de diseño de Meta4Objects sobre DTD/XML Schema

El diseño de Meta4Objects orientados a los procesos de transferencia de datos debe estar supeditado a las restricciones estructurales que impone la DTD o bien el XML Schema, tal y como se ha visto en los apartados anteriores. Sin embargo, el diseño de los Meta4Objects tendrá algunas particularidades, dependiendo de la funcionalidad que deben admitir, exportación o importación de datos.

Como apoyo a estos procesos, es necesario definir un conjunto de relaciones de correspondencia entre los elementos de la DTD/XML Schema y los elementos del Meta4Object.

Reglas de correspondencia entre un Meta4Object y una DTD/XML Schema

En la ejecución de los procesos de transferencia de datos, es recomendable especificar la correspondencia entre los elementos de la DTD o XML Schema y del Meta4Object. El procedimiento para establecer reglas de correspondencia entre elementos de la DTD/XML Schema y elementos del Meta4Object se hace invocando a una presentación específicamente desarrollada para este fin. Este proceso de correspondencia de elementos es el mecanismo que utiliza la aplicación para saber cómo manipular los datos.

Existen tres tipos de correspondencia:

- Correspondencia a nivel de Meta4Object
- Correspondencia a nivel de nodo
- Correspondencia a nivel de elementos

Correspondencia a nivel de Meta4Object

Seleccione la opción **Herramientas de desarrollo | Gestión de archivos XML | Correspondencia Meta4Object-DTD**.

A continuación dispone de una serie de campos que puede llenar de forma opcional a excepción del ID Meta4Object y el Tipo de correspondencia que son obligatorios.

Estos son los siguientes:

- **ID Meta4Object:** código identificativo del Meta4Object generado según la definición de una DTD/XML Schema.
- **Tipo de correspondencia:** valor numérico que identifica el tipo de asignación. Esto permite reutilizar un mismo Meta4Object en función de distintas asignaciones.
- **Nombre del fichero DTD/XML Schema:** nombre del archivo de definición de datos. Es necesario rellenar este campo con el nombre de la DTD o del XML Schema que se declara en el archivo XML que procesa los datos. En la cabecera del archivo XML debe aparecer el nombre de la DTD/XML Schema contra la que se quiere validar, que deberá coincidir con la que se asigne en este campo. Las formas de declaración de la DTD y del XML Schema dentro del archivo XML es de los siguientes tipos:

DTD:

```
<!DOCTYPE STD_PERSON SYSTEM "STD_PERSON.dtd">
```

XML SCHEMA sin namespaces:

```
<STD_PERSON xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:noNamespaceSchemaLocation="STD_PERSON.xsd">
```

XML SCHEMA con namespaces:

```
<STD_PERSON xmlns="http://www.meta4.com/schema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.meta4.com/schemas
STD_PERSON_ns.xsd">
```

- **Fichero DTD/XML Schema:** nombre del archivo de definición de datos propiamente dicho.
Tiene la opción entre elegir un archivo DTD con extensión .dtd, o bien un archivo XML Schema con extensión .xsd
- **Elemento raíz:** código identificativo del elemento raíz de la DTD/XML Schema.
- **Elemento base:** clave del elemento de la DTD/XML Schema que se toma como base del proceso de carga. Es el elemento que se toma como base en la carga parcial de registros en procesos de importación. En procesos de exportación, es el elemento que se toma como referencia para hacer el volcado de registros.
- **Clave de elemento base:** elemento o conjunto de elementos del Meta4Object que definen de forma unívoca en el Meta4Object una instancia del elemento base. En el caso de que se especifiquen distintos elementos, éstos aparecerán separados por comas (ID_COUNTRY,ID_PERSON....).

La presentación "Correspondencia Meta4Object-DTD" ofrece, para un Meta4Object determinado, un botón que carga los nodos que lo componen.

Herencia

La aplicación Meta4 permite realizar importaciones y exportaciones desde Meta4Objects heredados. A través de esta presentación, se obtiene información sobre el Meta4Object padre.

Correspondencia a nivel de nodo

Permite establecer relaciones de correspondencia definiendo qué elementos XML tienen correspondencia con qué nodos del Meta4Object.

En la opción **Herramientas de desarrollo | Gestión de archivos XML | Correspondencia Meta4Object-DTD**, seleccione la opción **Correspondencia de nodos**.

Rellene los siguientes campos:

- **ID Nodo:** código identificativo del nodo del Meta4Object.
- **Elemento XML:** elemento XML que corresponde con el nodo.
- **Orden del elemento XML:** orden que ocupa el elemento XML dentro de la declaración de contenido de un elemento compuesto. Por ejemplo:

```
<!ELEMENT REC_PERSON (STD_ID_PERSON, STD_N_FIRST_NAME,  
STD_N_FAMILY_NAME1, STD_N_USUAL_NAME?, STD_ID_GENDER?,  
STD_SALUTATION?, STD_DT_BIRTH?, STD_ID_COUNTRY?, STD_ID_GEO_DIV?,  
STD_ID_SUB_GEO_DIV?, STD_COMMENT?, SCO_PHOTO?, SCO_HOME_PAGE?,  
STD_ADDRESS*, STD_EMAIL*)>
```

El orden del elemento XML para el elemento STD_ADDRESS* será el número 14. Este campo no se rellena en el caso de que el elemento no forme parte de una secuencia. El orden de los elementos es especialmente importante en los procesos de exportación.

El botón **Cargar nodos** permite cargar automáticamente todos los nodos del Meta4Object y rellenar los valores correspondientes.

Correspondencia de elementos

Permite establecer relaciones de correspondencia definiendo qué elementos XML tienen correspondencia con qué elementos del Meta4Object.

En la opción **Herramientas de desarrollo | Gestión de archivos XML | Correspondencia Meta4Object-DTD**, seleccione la opción **Correspondencia de items**. El formulario se cargará con la correspondencia de elementos según el nodo que haya seleccionado en el formulario anterior.

Rellene los siguientes campos:

- **ID Item:** código identificativo del elemento en el Meta4Object.
- **Orden del elemento:** orden que ocupa el elemento XML dentro de una estructura de secuencia. El orden de los elementos es especialmente importante en la ejecución de los procesos de exportación.
- **Elemento XML:** identificador del elemento XML que tiene correspondencia con el elemento del Meta4Object.

- **Atributo de:** esta propiedad indica, sólo en caso de que un elemento tenga correspondencia con un atributo XML, el elemento XML del que es atributo.
- **ID Estructura de nodo padre:** código de identificación de la estructura del nodo padre del elemento
- **Valor del nodo:** marque esta casilla de verificación para definir un elemento como si fuera el valor del nodo al que pertenece. De esta forma se vuelcan los datos del elemento bajo la etiqueta XML definida para el nodo. Recuerde que sólo puede haber un elemento con esta marca y que el resto de elementos del nodo mapeados deben tener la propiedad "Atributo de" rellena.
- **Ignorar valores vacíos:** marque esta opción para que un elemento cuyo valor sea nulo no aparezca en el fichero XML de exportación, en lugar de que sea volcado al fichero XML como valor vacío (i.e. <fecha_nacimiento/>). Esto se requiere para la exportación XML con validación de esquemas, ya que los elementos de tipo numérico y fecha (xsd:int, xsd:date) no validan si se vuelcan valores vacíos. Tenga en cuenta que si trabaja con esquemas, para que la validación sea correcta los valores con esta marca deben tener en su definición la propiedad "mínimo de ocurrencias" igual a 0 (minocurrs="0").
- **Mapeos por defecto:** la definición de las reglas de correspondencia entre un Meta4Object y una DTD/XML Schema no es obligatoria. En los procesos de importación XML, para los Meta4Objects que no tengan correspondencia, se considerará que el nombre de los elementos XML son los nombres de los items de los nodos del Meta4Object. En los procesos de exportación XML, para los Meta4Objects que no tengan correspondencia, se considerará que el nombre de los items o de los nodos del Meta4Object es el nombre del elemento XML.

Mapeos por defecto sin persistencia

En el proceso de importación / exportación XML se pueden establecer mapeos por defecto mediante un API y después importar un fichero XML o exportar el Meta4Object a un fichero XML. Los mapeos por defecto se persisten en base de datos para que luego el método C++ pueda recuperar estos datos.

Sin embargo, existe otra forma de establecer mapeos por defecto automáticamente y es cuando se establece el argumento de entrada `maptype=nullvalue()`. En este caso, los mapeos no se persisten nunca.

Este procedimiento permite no persistir la información de mapeo en el caso de establecer los mapeos mediante el API exportado en el Meta4Object de mapeos `DMS_M4OBJEXT_XML_MAP` (`API_SET_DEFAULT_MAPPINGS`, etc.) y recuperar la instancia levantada previamente del Meta4Object de mapeos dentro del método C++ de importación XML y del método C++ de exportación XML.

Para ello, se recupera la instancia del Meta4Object a partir de su handle dentro del método C++, sin cargar el Meta4Object DMS_M4OBJECT_XML_MAP y sin realizar carga de datos alguna en este mismo Meta4Object (los datos ya han sido previamente cargados mediante el método de mapeos por defecto). Esto implica que no se llama al método FILTERED_LOAD desde la dll, simplemente se obtiene el sClChannel a partir del handle.

Para poder obtener el acceso al Meta4Object a partir del handle se ha implementado una nueva función en MV:

```
ClChannel* ClChannelManager :: GetChannelByHandle( m4int32_t  
ai_hChannel ) ;
```

A continuación se muestra un ejemplo. Tenemos un Meta4Object que se encarga de levantar por nivel dos el Meta4Object de mapeos DMS_M4OBJECT_XML_MAP y obtener su handle:

EJEMPLO PARA EL PROCESO DE IMPORTACIÓN XML

```
// Establecer los mapeos por defecto.  
DMS_M4OBJECT_XML_MAP!M4OBJECT_XML_MAP.API_SET_DEFAULT_MAPPINGS(sM4  
OName, iMapType, bPersist)  
  
// Obtenemos el handle de la instancia del Meta4Object de mapeos  
cargado.  
iChannelHandle = DMS_M4OBJECT_XML_MAP!GetHandle()  
  
// Importar el fichero XML en el Meta4Object de datos.  
DMST_DTD_PERSON!STD_PERSON.DLL_IMPORT_XML_BY_HANDLE(iMapType,  
sXMLFileName, bDoValidation, iFirstRegister, iLastRegister,  
iChannelHandle)
```

EJEMPLO PARA EL PROCESO DE EXPORTACIÓN XML

```
// Establecer los mapeos por defecto.  
DMS_M4OBJECT_XML_MAP!M4OBJECT_XML_MAP.API_SET_DEFAULT_MAPPINGS(sM4  
OName, iMapType, bPersist)  
  
// Obtenemos el handle de la instancia del Meta4Object de mapeos  
cargado.  
iChannelHandle = DMS_M4OBJECT_XML_MAP!GetHandle()  
  
// Importar el fichero XML en el Meta4Object de datos.  
bDoPublic = 0
```

```
DMST_DTD_PERSON!STD_PERSON.DLL_EXPORT_CHANNEL_BY_HANDLE(iMapType,
sXMLFileName, bDoValidation, sDtdFilePath, bDoPublic,
iChannelHandle)
```

Una vez obtenido el handle, se utilizará como argumento de entrada en dos nuevos métodos C++, que se mapean a un método LN4. Estos métodos se describen a continuación.

1. Para exportar un Meta4Object a un fichero XML utilizando un handle al Meta4Object DMS_M4OBJECT_XML_MAP:

Ítem método both, de nodo, de tipo dll que invoque a la componente C++ encargada de la carga de datos. (p.e. DLL_EXPORT_DATA_BY_HANDLE).

- **ID Recurso:** m4dmsxml
- **ID Método:** ExportXMLDataByHandle
- **Argumentos** (siguiendo el orden que se indica):
 1. **ARG_MAP_TYPE:**
 - Tipo: Número, Precisión = 1
 - Descripción: Tipo de correspondencia del Meta4Object con el DTD a tratar.
 2. **ARG_XML_FILE:**
 - Tipo: Cadena variable (255)
 - Descripción: Fichero XML donde van a ser volcados los datos del Meta4Object, ruta incluida. Al ser el método both, si se invoca en cliente será ruta de cliente y cuando se invoque en servidor será ruta de servidor.
 3. **ARG_DOVALIDATION:**
 - Tipo: Numérico , Precisión = 1
 - Descripción: Para indicar si se quiere generar el fichero XML con la información de validación del fichero XML con el DTD.
 - 0: No Validación, resto: Validación
 4. **ARG_DTD_PATH_FILE:**
 - Tipo: Cadena variable (255)
 - Descripción: Directorio donde se encuentra el DTD que se quiere incluir en el caso de especificar validación. Este campo sólo es aplicable si se especifica el argumento ARG_DOVALIDATION = 1, en otro caso no tiene sentido.
 5. **ARG_DOPUBLIC:**
 - Tipo: Numérico , Precisión = 1
 - Descripción: Para indicar si se quiere generar la validación del DTD con Public o System.

- 0: No Validación, resto: Validación

6. **ARG_CHANNEL_HANDLE:**

- Tipo: Número, Precisión = 18
- Descripción: handle del Meta4Object de mapeo.

2. Para importar un fichero XML utilizando un handle al Meta4Object
DMS_M4OBJECT_XML_MAP:

Ítem método both, de nodo, de tipo dll que invoque a la componente C++ encargada de la carga de datos. (p.e. DLL_IMPORT_DATA_XML_BY_HANDLE).

■ **ID Recurso:** m4dmsxml

■ **ID Método:** LoadXMLDataByHandle

■ **Argumentos** (siguiendo el orden que se indica):

1. **ARG_MAP_TYPE:**

- Tipo: Número, Precisión = 1
- Descripción: Tipo de Correspondencia del Meta4Object con el DTD a tratar.

2. **ARG_XML_FILE:**

- Tipo: Cadena variable (255)
- Descripción: Fichero XML a tratar (path) incluido. Al ser el método both, si se invoca en cliente será ruta de cliente y cuando se invoque en servidor será ruta de servidor.

3. **ARG_DOVALIDATION:**

- Tipo: Numérico , Precisión = 1
- Descripción: Para indicar si se quiere validación o no del fichero XML con el DTD.
- 0: No Validación, resto: Validación

4. **ARG_FIRST_REG:**

- Tipo: Numérico, Precisión = 1
- Descripción: Número del primer registro a tratar. La cuenta comienza desde 0. Si viene a nulo o con valor < 0, se considera que el primer registro es el cero.

5. **ARG_LAST_REG:**

- Tipo: Numérico, Precisión = 1
- Descripción: Último registro a tratar. -1 o Null implica tratar hasta el final.

6. **ARG_CHANNEL_HANDLE:**

- Tipo: Numérico, Precisión = 18
- Descripción: handle del M4Object de mapeos.

Estos dos métodos son iguales que los originales (LoadXML, ExportXMLData) pero se ha añadido un nuevo argumento de entrada para el handle al Meta4Object de mapeos. Es importante observar que el método para exportar ExportXMLData tiene un argumento variable (ARG_DOPUBLIC) que ahora ha pasado a ser fijo.

Durante el proceso de importación y exportación XML, cuando tenemos un handle, ya no se cargan datos en el Meta4Object DMS_M4OBJECT_XML_MAP (mediante el método FILTERED_LOAD), sino que se utilizan los que se han cargado previamente mediante el API_SET_DEFAULT_MAPPINGS. Al final de estos procesos, se descarga la información del Meta4Object de mapeos, independientemente de si se ha levantado el Meta4Object con el handle o no. Esto se hace llamando al método UNLOAD_INFO.

Respecto a la obtención del handle, es muy importante tener en cuenta que si el método de obtención del handle se ejecuta en cliente, el método de importación o exportación XML debe ejecutarse en cliente también para que el handle del Meta4Object sea válido. Lo mismo se aplica para el servidor. Es decir, que es esencial ejecutar ambos métodos en el mismo sitio.

XML y datos

La aplicación gestiona los procesos de importación y exportación de datos mediante archivos externos sujetos a la especificación XML. Actualmente no existen mecanismos automáticos para traspasar datos de Meta4 PeopleNet a archivos XML o viceversa. La forma de declaración de los archivos XML está supeditada a las restricciones implícitas de su propia DTD/XML Schema, y el mecanismo para anidar los distintos elementos estructurales cumple todas las especificaciones que indicó el consorcio W3C en la elaboración de este lenguaje de marcas.

Los Meta4Objects de las aplicaciones Meta4, previamente diseñados para admitir estos procesos, referencian a estos archivos externos a través de un conjunto de componentes que se registran con el propio paquete de instalación. Consulte los apartados *Meta4Objects en procesos de exportación* y *Meta4Objects en procesos de importación*.

EJEMPLO

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE STD_PERSON SYSTEM "C:TEMP\\STD_PERSON.dtd">
<STD_PERSON>
  <REC_PERSON>
    <STD_ID_PERSON>PERSON01</STD_ID_PERSON>
    <STD_N_FIRST_NAME>Patricia</STD_N_FIRST_NAME>
    <STD_N_FAMILY_NAME1>Fernandez</STD_N_FAMILY_NAME1>
    <STD_N_USUAL_NAME>Patricia</STD_N_USUAL_NAME>
    <STD_ID_GENDER>M</STD_ID_GENDER>
    <STD_SALUTATION>Mrs</STD_SALUTATION>
    <STD_DT_BIRTH>1975-10-01</STD_DT_BIRTH>
    <STD_ID_COUNTRY>Espana</STD_ID_COUNTRY>
    <STD_ID_GEO_DIV>Salamanca</STD_ID_GEO_DIV>
    <STD_ID_SUB_GEO_DIV>Ciudad Rodrigo</STD_ID_SUB_GEO_DIV>
    <STD_COMMENT/>
    <SCO_HOME_PAGE>http://patricia.meta4.es</SCO_HOME_PAGE>
    <STD_ADDRESS>
      <STD_ID_PERSON>PERSON01</STD_ID_PERSON>
      <STD_OR_ADDRESS>Roma 3, Esc2 1A</STD_OR_ADDRESS>
      <STD_ADDRESS_LINE_1>Roma 3</STD_ADDRESS_LINE_1>
      <STD_ADDRESS_LINE_2>Escalera 2</STD_ADDRESS_LINE_2>
      <STD_ADDRESS_LINE_3>Piso 1</STD_ADDRESS_LINE_3>
      <STD_ADDRESS_LINE_4>Puerta A</STD_ADDRESS_LINE_4>
```

```

<STD_ID_COUNTRY>Espana</STD_ID_COUNTRY>
<STD_ID_GEO_DIV>Salamanca</STD_ID_GEO_DIV>
<STD_ID_SUB_GEO_DIV>Tormes</STD_ID_SUB_GEO_DIV>
<STD_ID_GEO_PLACE>12</STD_ID_GEO_PLACE>
<STD_ZIP_CODE>47678</STD_ZIP_CODE>
</STD_ADDRESS>

```

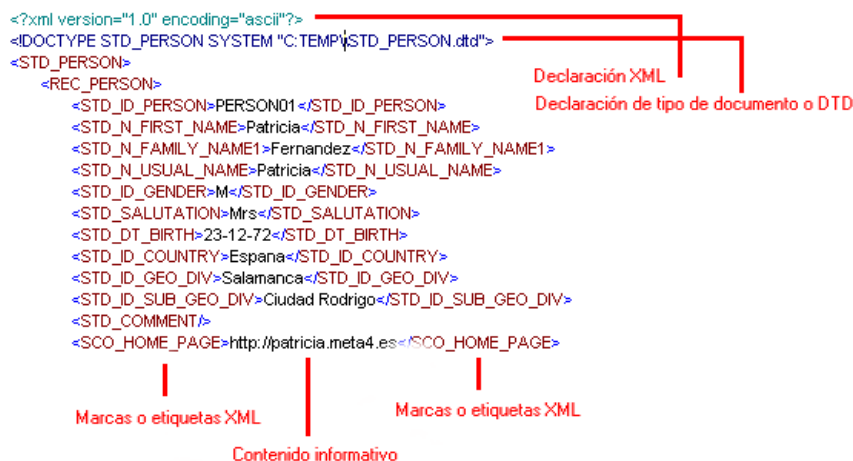
¿Cómo interpretar un archivo XML en procesos de importación o exportación desde una aplicación Meta4?

Un archivo XML es un conjunto de elementos anidados que están organizados bajo una estructura sintáctica en forma de árbol. El prólogo a la estructura básica del documento XML está formado por la declaración XML y la declaración de tipo de documento, aunque este último parámetro es opcional y depende de si se está validando contra una DTD/XML Schema. La declaración XML permite saber a la aplicación encargada de procesar el documento XML que se trata de un documento de este tipo. La declaración de tipo de documento o DTD/XML Schema contiene las reglas sintácticas que deben cumplir los documentos de un tipo determinado.

Tras la declaración XML y la de contenido, aparecen definidos los distintos elementos. Éstos están constituidos tanto por las marcas para identificar los elementos estructurales como por el contenido informativo de los mismos.

Un elemento en un archivo XML está comprendido por: *las marcas*, fácilmente identificables porque se encuentran precedidas del carácter menor que (<), y seguidas por el carácter mayor que (>); y por *el contenido* normalmente escrito dentro de estas dos etiquetas.

Figura 2. Estructura de un archivo XML



El contenido informativo constituye realmente el conjunto de datos que se procesan durante la importación y exportación.

Para que la aplicación sea capaz de interpretar las marcas que identifican a cada elemento y deducir la estructura jerárquica del documento XML, es necesario que las marcas estén anidadas correctamente.

Se pueden aplicar las siguientes reglas:

- La marca de fin de un elemento A no puede escribirse hasta que no se hayan introducido todas las etiquetas de cierre de los elementos cuya marca de inicio esté situada después de la marca de apertura del elemento A”.
- No se puede introducir una marca de cierre de un elemento si no se ha introducido antes su marca de inicio.
- Para todos los elementos no vacíos del documento se debe incluir una marca de inicio y una de fin. Por ejemplo: <STD_COMENTARIO/> o <STD_COMENTARIO> </STD_COMENTARIO>.

Meta4Objects en procesos de importación

Los procesos de importación en las aplicaciones Meta4 están diseñados para procesar modelos de transferencia de datos desde archivos externos basados en XML estándar.

En el procesamiento de los datos, éstos se cargarán en un Meta4Object que hará las veces de destino de la importación y, simultáneamente, de origen de la llamada al archivo XML externo.

Previamente a la ejecución de los procesos de importación, será necesario adecuar los entornos para garantizar el correcto procesamiento de la transferencia de datos.

Requerimientos previos a la fase de importación

Existe un conjunto de requerimientos y recomendaciones que se deben tener en cuenta antes de ejecutar procesos de importación.

Así, es recomendable realizar un análisis pormenorizado de la estructura de los datos que se quieren procesar en la importación. La estructura de los datos, o contenido informativo del documento, se almacena en un archivo con extensión .xml. En la actualidad, el mercado ofrece distintas herramientas que permiten procesar archivos de este tipo. Obviamente, los datos que se procesan en la importación se encuentran organizados y anidados en función de una estructura sintáctica determinada por la DTD/XML Schema. El diseño de la DTD/XML Schema debe cumplir un doble objetivo: por un lado, determina los elementos estructurales (etiquetas o marcas), que aparecen declarados en el archivo XML que contiene los datos procesables; por otro lado, determina el procedimiento de diseño de los distintos elementos de un Meta4Object desde los que se van a importar los datos.

En resumen, es necesario diferenciar los metadatos (DTD/XML Schema y Meta4Objects) y los datos (archivos .xml), y ser conscientes de las restricciones implícitas que imponen los primeros respecto a los segundos.

Descripción del proceso de importación

La aplicación proporciona los mecanismos necesarios para que, dado un archivo XML, una DTD/XML Schema y un Meta4Object destino con las restricciones propias que debe cumplir cada elemento, se pueda procesar una importación de datos.

El proceso de importación se encarga básicamente de procesar el contenido informativo del archivo XML y volcar los datos sobre la estructura de un

Meta4Object. La aplicación gestiona un proceso interno de validaciones para garantizar el correcto diseño de los distintos elementos. El proceso de validaciones se apoya en una biblioteca (m4dmsxml.dll) específicamente desarrollada para gestionar el algoritmo de importación. Aunque este componente se descarga con el propio paquete de instalación, deberá asegurarse de que está perfectamente instalado en su sistema.

El algoritmo de la biblioteca admite lecturas completas o parciales de todos los registros de un archivo XML. Además, se encarga de validar y realizar las siguientes comprobaciones:

- Valida la existencia del archivo XML origen y del Meta4Object destino.
- Comprueba la validez del archivo XML respecto a la DTD/XML Schema (sólo en el caso de que esté validando contra ésta). Este proceso sólo se lleva a cabo si el usuario ha especificado como argumento de entrada al proceso de importación XML ARG_DO_VALIDATION = 1.
- Garantiza que los elementos estructurales del Meta4Object (nodos o elementos) corresponden con los declarados en la DTD/XML Schema.
- Realiza comprobaciones de obligatoriedad de elementos
- Mantiene un control para determinar concretamente dónde se han producido errores o inconsistencias durante el proceso.

¿Cómo invocar a la biblioteca desde un Meta4Object y cómo declarar la llamada a un archivo XML?

Los objetos de Meta4 PeopleNet preparados para admitir procesos de importación de datos deben tener las mismas restricciones estructurales que la DTD/XML Schema sobre la que han basado su diseño; pero, además, deben incluir algunos métodos que permiten inicializar el proceso de transferencia de datos e invocar al archivo XML origen de los mismos.

Sobre el Meta4Object destino de la importación es necesario crear dos métodos (con ámbito a nivel de nodo) sobre el nodo raíz: uno que instancia a la biblioteca m4dmsxml.dll y otro en el que se asignan valores a los parámetros declarados en el método anterior. Entre ambos métodos existe una relación de dependencia interna, puesto que desde la regla de uno de ellos se invoca al otro.

Por ejemplo, el método que instancia la biblioteca se denomina DLL_XML_IMPORT; y el método que invoca al archivo XML se llama IMPORT.

El método DLL_XML_IMPORT debe tener una regla en donde se referencie la biblioteca que admite el proceso de validaciones de la importación, y cinco argumentos que se le pasan como parámetros. Estos parámetros se definen desde la pestaña **Argumentos** en el Diseñador de Meta4Objects, y se les asignan valores desde el otro método (IMPORT).

Las propiedades de los argumentos son:

Tabla 3. Propiedades de los argumentos en importación

ID Argumento	Nombre argumento	Posición	Tipo argumento	Tipo básico
ARG_MAP_TYPE	ARG_MAP_TYPE	0	Valor	Número
ARG_XML_FILE	ARG_XML_FILE	1	Valor	Long
ARG_DO_VALIDATION	ARG_DO_VALIDATION	2	Valor	Número
ARG_FIRST_REGISTER	ARG_FIRST_REGISTER	3	Valor	Número
ARG_LAST_REGISTER	ARG_LAST_REGISTER	4	Valor	Número

Estos argumentos representan lo siguiente:

- **ARG_MAP_TYPE:** tipo de correspondencia del Meta4Object con la DTD/XML Schema por procesar.
- **ARG_XML_FILE:** ruta física (del cliente o del servidor) del archivo XML origen de los datos (por ejemplo, sXMLFileName = "C:\XMLMIND\STD_PERSON.xml").
- **ARG_DO_VALIDATION:** indica si se quiere validar estructuralmente el archivo XML respecto de su DTD/XML Schema. Si se asigna el valor 0 no se valida nada; si se asigna cualquier otro valor distinto, el algoritmo procesa la validación.
- **ARG_FIRST_REGISTER/ARG_LAST_REGISTER:** número del primer o último registro por procesar. Indica si aplica un procesamiento parcial de carga de registros sobre el archivo XML. El procesamiento completo de los registros se produce si estos parámetros reciben los valores *nullvalue()*.

A continuación debe rellenar las propiedades de la regla del método, desde el formulario **Editor de fórmulas**:

Los campos son los siguientes:

- **Tipo de código:** tipo de código asociado a la regla, es decir, mecanismo que utiliza la aplicación para hacer la llamada a la ejecución de la regla. Esta llamada se hace a través de una DLL o biblioteca de enlace dinámico.
- **Recurso:** nombre de la DLL que se ve afectada por la llamada (m4dmsxml).
- **Método:** nombre de la función que utiliza la DLL para hacer la llamada a la ejecución de la regla (LoadXMLData y LoadXMLDataExtended).

El método IMPORT, por el contrario, debe tener una regla e incluir en la declaración de la fórmula de su regla el siguiente código (tipo LN4):

```

*****
' Fecha:
' Version PeopleNet en la que aparece: 3.2
' Ejemplo de pruebas importaciones de datos.
'
*****

iMapType = 1
sXMLFileName = "Ruta física del fichero XML"
bDoValidation = 1
iFirstRegister = nullvalue()
iLastRegister = nullvalue()

X = DLL_IMPORT_XML(iMapType, sXMLFileName, bDoValidation,
iFirstRegister, iLastRegister)
return X

```

A través de esta función se asignarán valores a los argumentos declarados en el método anterior. Los valores asignados a los parámetros determinan el tratamiento en la carga parcial de registros, procesos de validación, etcétera:

Los Meta4Objects diseñados con las propiedades específicas de este tipo de métodos, además de las restricciones que se han señalado a lo largo del documento, cumplen los requerimientos necesarios para admitir procesos de importación de datos. Existen alternativas válidas desde el punto de vista de diseño que evitan la duplicación de estos métodos en los Meta4Objects que vayan a ejecutar procesos de este tipo. Aunque no se va a analizar detalladamente otro tipo de alternativa, se recomienda diseñar estructuras de nodo con la propiedad y los métodos de carga de datos XML, y a continuación diseñar Meta4Objects con, al menos, un nodo libre que instancie la estructura de nodo generada.

Adicionalmente, se dispone de un método que añade la funcionalidad de los XML Schemas. A este método se le denomina LoadXMLDataExtended y permite importar un archivo XML, validar contra una DTD/XML Schema y especificar un handle al Meta4Object de mapeos para poder cargar una instancia previamente levantada.

Las propiedades de los argumentos para el método LoadXMLDataExtended son:

Tabla 4. Propiedades de los argumentos en importación método LoadXMLDataExtended

ID Argumento	Nombre argumento	Posición	Tipo argumento	Tipo básico
ARG_MAP_TYPE	ARG_MAP_TYPE	0	Valor	Número

Tabla 4. Propiedades de los argumentos en importación método LoadXMLDataExtended

ID Argumento	Nombre argumento	Posición	Tipo argumento	Tipo básico
ARG_XML_FILE	ARG_XML_FILE	1	Valor	Long
ARG_DO_VALIDATION	ARG_DO_VALIDATION	2	Valor	Número
ARG_FIRST_REGISTER	ARG_FIRST_REGISTER	3	Valor	Número
ARG_LAST_REGISTER	ARG_LAST_REGISTER	4	Valor	Número
ARG_CHANNEL_HANDLE	ARG_CHANNEL_HANDLE	5	Valor	Número
ARG_USE_XML_SCHEMA	ARG_USE_XML_SCHEMA	6	Valor	Número

Estos argumentos representan lo siguiente:

- ARG_XML_FILE: ruta física (del cliente o del servidor) del archivo XML origen de los datos (por ejemplo, sXMLFileName = "C:\XMLMIND\STD_PERSON.xml").
- ARG_DO_VALIDATION: indica si se quiere validar estructuralmente el archivo XML respecto de su DTD/XML Schema. Si se asigna el valor 0 no se valida nada; si se asigna cualquier otro valor distinto, el algoritmo procesa la validación.
- ARG_FIRST_REGISTER/ARG_LAST_REGISTER: número del primer o último registro por procesar. Indica si aplica un procesamiento parcial de carga de registros sobre el archivo XML. El procesamiento completo de los registros se produce si estos parámetros reciben los valores *nullvalue()*.
- ARG_CHANNEL_HANDLE: handle al M4Object de correspondencia. Se permite especificar el handle a una instancia del M4Object DMS_M4OBJECT_XML_MAP con información de correspondencia definida por el usuario.
- ARG_USE_XML_SCHEMA: este argumento se usa para indicar si la validación se realiza contra un XML Schema. Los posibles valores son:
 - 1: usar XML Schema
 - Resto: usar DTD

Procedimiento para ignorar elementos XML sin correspondencia

El proceso de importación XML realiza la carga de datos de un fichero XML en un Meta4Object. Este proceso utiliza una correspondencia entre los objetos XML y los nodos/elementos del Meta4Object (mapeo), que puede ser generada

por el usuario o bien generarse automáticamente, es decir, cada elemento XML debe tener un nodo/elemento con el mismo identificador en el Meta4Object.

Para hacer que la importación XML sea menos restrictiva, se ha añadido un parámetro a la función LoadXMLDataExtended, que permite ignorar los elementos XML que no tienen correspondencia con los elementos mapeados del Meta4Object, sin que el proceso devuelva un error.

Tenga en cuenta que si un elemento debe ser ignorado, todos sus atributos y elementos hijos se ignorarán. Sin embargo, si en el mapeo del Meta4Object se define un elemento base y/o un elemento raíz, estos no pueden ser ignorados.

En el proceso de importación, cuando se ha configurado la función LoadXMLDataExtended, para que ignore los elementos sin mapeo, hay que pasarle un nuevo argumento opcional que indica este comportamiento. Si se pasa a True, valor por defecto, se verificará la existencia de los elementos XML en los elementos del Meta4Object mapeado. Si se pasa a False, no se verificará la existencia de los elementos XML en el Meta4Object mapeado.

Meta4Objects en procesos de exportación

Los procesos de exportación en las aplicaciones Meta4 están diseñados para permitir exportar datos procesables en la aplicación a archivos externos basados en XML estándar.

En el procesamiento de la exportación de datos, el sistema ejecuta un algoritmo para extraer los datos almacenados en el Meta4Object y transformar éstos en elementos procesables por archivos XML.

La principal diferencia entre este proceso y el de importación de datos, es que en el primero no es necesario un archivo XML con los datos, como sucedía en el segundo, sino que es el propio proceso el que infiere éstos desde la propia estructura y datos del Meta4Object.

Previamente a la ejecución de los procesos de exportación, será necesario adecuar los entornos para garantizar el correcto procesamiento de la transferencia de datos. En cualquier caso, los requerimientos desde el punto de vista de diseño en los procesos de importación y exportación difieren exclusivamente en la parametrización de los Meta4Objects respectivos. La definición de Meta4Objects en función de la DTD/XML Schema, y las reglas de correspondencia de elementos señaladas a lo largo del documento, son requisitos imprescindibles en cualquiera de los dos procesos.

Requerimientos previos a la fase de exportación

Existen unos requerimientos y recomendaciones que se deben tener en cuenta antes de ejecutar procesos de exportación.

Los datos que se procesan en la exportación se encuentran encapsulados en la estructura del Meta4Object. La disposición estructural del Meta4Object de exportación cumple las restricciones sintácticas determinadas por la DTD/XML Schema, y determinará el mecanismo en el que se anidarán las etiquetas y el contenido informativo de los datos procesados desde el Meta4Object.

Una vez procesada la exportación de datos, en la aplicación no existe ningún mecanismo para validar que el documento XML está bien formado, es decir, que todas sus etiquetas se encuentran correctamente anidadas. Existen editores XML convencionales que validan este tipo de comportamiento. En el caso de que se detecten errores de este tipo después de lanzar un proceso de exportación de datos, será necesario comprobar que tanto el Meta4Object como su DTD/XML Schema se encuentran bien contruidos, y que si hay mapeos, estos están bien definidos.

Descripción del proceso de exportación

La aplicación proporciona los mecanismos necesarios para que, dados una DTD/XML Schema y un Meta4Object origen, se pueda procesar una exportación de datos con las restricciones propias que debe cumplir cada elemento.

El proceso de exportación se encarga básicamente de exportar los datos almacenados en un Meta4Object y transformarlos en un archivo XML. El algoritmo de ejecución del proceso de exportación se basa en la misma biblioteca que en el caso del proceso de importación, m4dmsxml.dll.

A diferencia del proceso de importación, el algoritmo de exportación no realiza validaciones ni comprobaciones de integridad para garantizar la correcta parametrización del entorno. No se detectarán inconsistencias entre Meta4Objects o DTD/XML Schema, o entre el archivo XML y la DTD/XML Schema, etcétera, hasta que el proceso de exportación haya finalizado.

¿Cómo invocar a la biblioteca desde un Meta4Object en procesos de exportación?

Un Meta4Object preparado para mantener procesos de exportación, además de cumplir las restricciones de la DTD/XML Schema, debe incluir un conjunto de métodos que permitan iniciar el proceso de exportación.

En el Meta4Object de exportación es necesario crear dos métodos (con ámbito a nivel de nodo) sobre el nodo raíz: uno que instancia a la biblioteca m4dmsxml.dll, y otro en el que se asignan valores a los parámetros declarados en el método anterior. Entre ambos métodos existe una relación de dependencia interna, puesto que desde la regla de uno de ellos se invoca al otro.

Por ejemplo, el método que instancia la biblioteca se denomina DLL_XML_EXPORT y el que invoca al archivo XML se llama EXPORT_CHANNEL.

El método DLL_XML_EXPORT debe tener una regla en la que se reference la biblioteca que mantiene el proceso de validaciones de la exportación, y cuatro argumentos que se le pasan como parámetros. Éstos se definen desde la pestaña **Argumentos** en el Diseñador de Meta4Objects, y se les asigna valores desde el otro método (EXPORT_CHANNEL).

Las propiedades de los argumentos para este método son:

Tabla 5. Propiedades de los argumentos en exportación

ID Argumento	Nombre argumento	Posic.	Tipo argumento	Tipo básico
ARG_MAP_TYPE	ARG_MAP_TYPE	1	Valor	Número
ARG_XML_RESULT_FILE	ARG_XML_RESULT_FILE	2	Valor	Long
ARG_USE_VALIDATION	ARG_USE_VALIDATION	3	Valor	Número
ARG_DTD_FILE_PATH	ARG_DTD_FILE_PATH	4	Valor	Long

Estos argumentos representan lo siguiente:

- **ARG_MAP_TYPE:** tipo de correspondencia del Meta4Object con la DTD/XML Schema que se va a procesar.
- **ARG_XML_RESULT_FILE:** ruta física (del cliente o del servidor) del archivo XML donde se van a volcar los datos procesados por la exportación (por ejemplo, sXMLFileName = "C:\TEMP\STD_PERSON.xml").
- **ARG_USE_VALIDATION:** indica si se debe volcar en el fichero XML la referencia al archivo DTD/XML Schema señalado durante el proceso de correspondencia entre el DTD/XML Schema y el Meta4Object.
- **ARG_DTD_FILE_PATH:** ruta física donde se encuentra la DTD/XML Schema que se quiere validar. Esta propiedad sólo tiene sentido si se especifica el argumento iDoValidation con el valor 1.

A continuación debe rellenar las propiedades de la regla del método.

- **Tipo de código:** tipo de código asociado a la regla, es decir, mecanismo que utiliza la aplicación para hacer la llamada a la ejecución de la regla. Esta llamada se hace a través de una DLL o biblioteca de enlace dinámico.
- **Recurso:** nombre de la DLL que se ve afectada por la llamada (m4dmsxml).
- **Método:** nombre de la función que utiliza la DLL para hacer la llamada a la ejecución de la regla (ExportXMLData o ExportXMLDataExtended).

El método EXPORT_CHANNEL, por el contrario, debe tener una regla e incluir en la declaración de la fórmula de su regla el siguiente código (tipo LN4):

```
*****
' Fecha:
' Version PeopleNet en la que aparece: 3.2
' Descripción: Ejemplo de pruebas exportaciones de datos
```

```

'
*****
// Método de exportación de Meta4Objects //
iMapType = 1
sXmlFile = "Ruta física fichero XML"
iDoValidation = 1
sDtdFilePath = "Ruta física DTD"
RC = M4_SUCCESS

return RC

```

A través de esta función, se asignarán valores a los argumentos declarados en el método anterior:

Adicionalmente, para validar una exportación contra un XML Schema se cuenta con el método ExportXMLExtended. Este método permite exportar un M4Object a un archivo XML y especificar un handle al M4Object de mapeos para que se cargue una instancia previamente levantada.

Las propiedades de los argumentos para el método ExportXMLExtended son:

Tabla 6. Propiedades de los argumentos en exportación método ExportXMLExtended

ID Argumento	Nombre argumento	Posic.	Tipo argumento	Tipo básico
ARG_MAP_TYPE	ARG_MAP_TYPE	1	Valor	Número
ARG_XML_RESULT_FILE	ARG_XML_RESULT_FILE	2	Valor	Long
ARG_USE_VALIDATION	ARG_USE_VALIDATION	3	Valor	Número
ARG_VALIDATION_FILE_PATH	ARG_VALIDATION_FILE_PATH	4	Valor	Long
ARG_DO_PUBLIC	ARG_DO_PUBLIC	5	Valor	Número
ARG_CHANNEL_HANDLE	ARG_CHANNEL_HANDLE	6	Valor	Número
ARG_USE_XML_SCHEMA	ARG_USE_XML_SCHEMA	7	Valor	Número
ARG_XML_SCHEMA_NAMESPACE	ARG_XML_SCHEMA_NAMESPACE	8	Valor	Long

Estos argumentos representan lo siguiente:

- ARG_MAP_TYPE: tipo de correspondencia del Meta4Object con la DTD/XML Schema que se va a procesar.
- ARG_XML_RESULT_FILE: ruta física (del cliente o del servidor) del archivo XML donde se van a volcar los datos procesados por la exportación (por ejemplo, sXMLFileName = "C:\TEMP\STD_PERSON.xml").
- ARG_USE_VALIDATION: indica si se debe volcar en el fichero XML la referencia al archivo DTD/XML Schema señalado durante el proceso de correspondencia entre el DTD/XML Schema y el Meta4Object.
- ARG_VALIDATION_FILE_PATH: directorio donde se encuentra el archivo de validación (DTD o XML Schema) que se quiere incluir en el caso de especificar validación. Este campo sólo es aplicable si se especifica el argumento ARG_DO_VALIDATION = 1; en otro caso no tiene sentido.
- ARG_DO_PUBLIC: este argumento sirve para indicar que se quiere establecer en el DOCTYPE del DTD el valor público a través de una URL. Los posibles valores son:
 - 1: PUBLIC
 - Resto: SYSTEM
- ARG_CHANNEL_HANDLE: handle al M4Object de correspondencia. Se permite especificar el handle a una instancia del M4Object DMS_M4OBJECT_XML_MAP con información de correspondencia definida por el usuario.
- ARG_USE_XML_SCHEMA: este argumento se usa para indicar que se quiere validar contra un XML Schema. Los posibles valores son:
 - 1: usar XML Schema
 - Resto: usar DTD
- ARG_XML_SCHEMA_NAMESPACE: en el caso de usar un XML Schema en la validación, especifica el Namespace utilizado en el XML Schema. No es obligatorio tener un Namespace en el Schema, pero de ser así, hay que especificarlo.